Reporte del trabajo grupal — Abarrotes Verde

Fecha del trabajo: 22 de octubre de 2026. Reporte preparado el 23 de octubre de 2026.

Mi participación

Mi rol fue crear los custom hooks useDebounce y useApi, siguiendo el ejemplo de la hermana Belén. Trabajé en feat/custom-hooks. Los dos archivos se entregaron en el PR #1 y el equipo ya los integró a main.

- useDebounce<T> espera 3000 ms después del último cambio. Usa useState, useEffect, setTimeout y clearTimeout. El input cambia inmediatamente; el filtro usa el valor con pausa.
- useApi<T> recibe una función que devuelve Promise<T>. Entrega data, cargando, error y recargar. Usa then, catch y finally. Al recargar aumenta un contador que vuelve a ejecutar el efecto. Su cleanup evita actualizar el estado con respuestas anteriores; no cancela físicamente el HTTP.

Cómo se conecta mi parte

La interfaz importa useDebounce y filtra con el texto que devuelve. El servicio que se pase a useApi debe devolver los datos, por ejemplo respuesta.data. Después de crear, editar o eliminar, se llama a recargar(). Esta versión de clase consulta al montar y al recargar; cambiar filtros no dispara una nueva consulta automáticamente.

Trabajo del equipo integrado

La versión recibida de main incluye la tienda, navegación, catálogo, filtros por categoría, carrito y layout de administración. Es trabajo del equipo, no solo de mi rol. Para preparar la publicación incorporé esos cambios a mi rama sin modificar main.

El catálogo usa productos locales de ejemplo. El login y otras secciones aún muestran pantallas pendientes. El hook useApi está disponible para cuando se conecten los servicios; actualmente el catálogo no lo utiliza.

Preparación de la publicación desde mi rama

Se conectó useDebounce al buscador existente del catálogo. Se corrigió un error de TypeScript usando el mensaje de confirmación al agregar al carrito. Se configuró la ruta base de GitHub Pages y navegación con hash para evitar errores al recargar rutas publicadas.

El workflow compila la web desde feat/custom-hooks, adjunta la web y este reporte como artefacto y publica en Pages si el propietario ya activó GitHub Actions como origen. No modifica main.

GitHub Pages todavía requiere activación por parte del propietario del repositorio. Mi cuenta tiene permiso para subir código, pero no para cambiar Settings > Pages. La web no debe darse por publicada hasta que termine el job deploy.

Comprobación

TypeScript: npx tsc -b. El build completo se verifica en el workflow de GitHub Actions sobre Linux; la ejecución local de Vite está bloqueada por las dependencias nativas en Windows. El resultado del workflow es la referencia para la compilación de esta entrega.

PR de hooks: https://github.com/adanbnjm/ecommerce-frontend/pull/1
Rama: https://github.com/adanbnjm/ecommerce-frontend/tree/feat/custom-hooks